

Pseudo-code for the 'test_closedloop.ipynb' file

Davide Carecci; reserved for A2A S.p.A.

January 9, 2026

In the following, ***bold*** refers to vectorization in the variables spaces, whereas the time vectorization is implicit. When time indexing is relevant, it is explicitly reported as subscript. This pseudo-code is intended to support and guide the reading and user experience of the 'test_closedloop.ipynb', that calls for the 'RPI_selectorPI_operative.py' and 'test_closedloop_integrator.py' functions to simulate a closed-loop experiment using the 'BIOGoAIS_agri-AcoDM' tool as real plant emulator.

Nomenclature

$(x_{C,gb}/x_{M,gb})_{ub,high}$	Higher hysteresis' activation threshold for $x_{C,gb}/x_{M,gb}$ upper bound.
$(x_{C,gb}/x_{M,gb})_{ub,low}$	Lower hysteresis' activation threshold for $x_{C,gb}/x_{M,gb}$ upper bound.
$\theta_{\text{integrator}}$	List of integration hyper-parameters (e.g. start/end timestamps, integration time step Δ_t).
χ_{danger}	Boolean variable that points out the safety condition of the working point.
$\tilde{\mathbf{y}}$	<i>Online</i> (automatic, high-frequency) measured data, already pre-processed.
$\mathbf{d}_{\text{ref}}$	Disturbances i.e. flow rates of non-controllable feedstocks as optimized by the agri-AcoDM's <i>off-line</i> optimization.
$\mathbf{e}_{\mathbf{y}}$	Output tracking errors.
$\mathbf{T}$	Vector of time instants to perform integration over.
$\mathbf{x}$	Agri-AcoDM non-(continuously)-measurable state vector.
$\mathbf{y}_{\text{ref}}$	Output setpoints set by agri-AcoDM.
$q_{\tilde{M},gb}$	Measured methane flow rate (in L h ⁻¹).
$q_{\tilde{TOT}}$	Measured total biogas flow rate (in L h ⁻¹).
$x_{C,gb}/\tilde{x}_{M,gb}$	Measured biogas composition ratio (dimensionless).
$x_{\tilde{C},gb}$	Measured CO ₂ volume fraction in biogas (dimensionless).
$x_{\tilde{M},gb}$	Measured CO ₂ volume fraction in biogas (dimensionless).

I	Accumulated integral term of the PI controller.
k	Index of the control run.
k_P	Proportional gain of the PI controller.
T_c	Control interval.
T_I	Integral time constant of the PI controller (in seconds).
u	Feedback correction/deviation of the control action computed by the controller.
u_0	Equilibrium input/feedforward control action of the controller.
u_I	Integral component of the control action of the PI controller.
u_P	Proportional component of the control action of the PI controller.
u_{actual}	Actual actuator command.
u_{max}	Upper bound saturation of u .
u_{min}	Lower bound saturation of u .
var^+	Updated value of the var variable.
var^-	"Preliminary" computation for the value of the var variable.
$var^{(i)}$	i^{th} entry of the var vector.
var_k	Value at the k^{th} control run of the var variable. Also as $var_{sub} _k$ for the var_{sub} variable.

Pseudo-code

Algorithm 1 main.py

```

1: for  $k$  in range(0,  $T_c$ , end of experiment) do
2:   Initialize Logger
3:   Round the timestamp correspondent to  $k$  to HH:00:00 (now)
4:   Read the CSV file containing  $\tilde{\mathbf{y}} = [q_{T\tilde{O}T}, x_{M,gb}, x_{C,gb}]$ 
5:   Filter  $\tilde{\mathbf{y}}$  between start to end timestamps (end = now)
6:   Compute  $q_{M,gb}$  and  $x_{C,gb}/x_{M,gb}$ . Extract  $q_{M,gb}|_k$  and  $(x_{C,gb}/x_{M,gb})_k$ 
7:   Read the TXT file containing  $\mathbf{y}_{\text{ref}} = [q_{M,gb,ref}, (x_{C,gb}/x_{M,gb})_{ref}]$  (setpoints set
   by agri-AcoDM),  $(x_{C,gb}/x_{M,gb})_{ub,low}$  and  $(x_{C,gb}/x_{M,gb})_{ub,high}$ . Extract  $q_{M,gb,ref}|_k$  and
    $(x_{C,gb}/x_{M,gb})_{ref}|_k$ 
8:   Save  $\tilde{\mathbf{y}}|_k$  and  $\mathbf{y}_{\text{ref}}|_k$  to CSV file
9:   Compute  $\mathbf{e}_y|_k = [q_{M,gb}|_k - q_{M,gb,ref}|_k, (x_{C,gb}/x_{M,gb})_k - (x_{C,gb}/x_{M,gb})_{ref}|_k]$ 
10:  Instantiate PICONROLLER class of the SELECTORPI_CONTROLLER.PY file as header ( $\rightarrow$ 
    $q_{M,gb}$  PI controller) and follower ( $\rightarrow$   $x_{C,gb}/x_{M,gb}$  PI controller)
11:  procedure OBJECT.LOAD_STATE(date, filename) for OBJECT  $\in [header, follower]$ 
12:    return  $I_{k-1}, \chi_{\text{danger}}|_{k-1}$ 
13:  end procedure
14:  Instantiate HYSTERESISCOMPARATOR class of the SELECTORPI_CONTROLLER.PY file
15:  procedure HYSTERESISCOMPARATOR.UPDATE( $(x_{C,gb}/x_{M,gb})_k, \chi_{\text{danger}}|_{k-1}$ )
16:     $\chi_{\text{danger}}|_k = \text{not } \chi_{\text{danger}}|_{k-1} \text{ and } (x_{C,gb}/x_{M,gb})_k > (x_{C,gb}/x_{M,gb})_{ub,high} \text{ or } \chi_{\text{danger}}|_{k-1}$ 
    and  $(x_{C,gb}/x_{M,gb})_k \geq (x_{C,gb}/x_{M,gb})_{ub,low}$ 
17:    return  $\chi_{\text{danger}}|_k$ 
18:  end procedure
19:  if  $\chi_{\text{danger}}|_k \neq \chi_{\text{danger}}|_{k-1}$  and not  $\chi_{\text{danger}}|_{k-1}$  then
20:    Compute  $I_{k-1}^+ = header.u - follower.k_P \cdot e_y^{(0)}|_k \cdot \frac{follower.T_I}{follower.k_P}$ 
21:    procedure follower.RESET_STATE( $I_{k-1}^+$ )
22:      Override  $I_{k-1}$  with  $I_{k-1}^+$ 
23:    end procedure
24:  else if  $\chi_{\text{danger}}|_k \neq \chi_{\text{danger}}|_{k-1}$  and  $\chi_{\text{danger}}|_{k-1}$  then
25:    Compute  $I_{k-1}^+ = follower.u - header.k_P \cdot e_y^{(1)}|_k \cdot \frac{header.T_I}{header.k_P}$ 
26:    procedure header.RESET_STATE( $I_{k-1}^+$ )
27:      Override  $I_{k-1}$  with  $I_{k-1}^+$ 
28:    end procedure
29:  else break
30:  end if
31:  procedure OBJECT.COMPUTE(error $|_k$ ) for OBJECT  $\in [header, follower]$ , error  $\in [e_y^{(0)}, e_y^{(1)}]$ 
32:     $u_P|_k = k_P \cdot \text{error}$ 
33:     $I_k = I_{k-1} + T_c \cdot \text{error}$ 
34:     $u_I|_k = I_k \cdot \frac{k_P}{T_I}$ 
35:     $u_k^- = u_P|_k + u_I|_k$ 
36:    Apply saturation:  $u_k = \text{if } u_k^- \leq u_{min} \text{ then } u_{min} \text{ else if } u_k^- \geq u_{max} \text{ then } u_{max} \text{ else } u_k^-$ 
37:    Apply anti-windup:  $I_k^+ = \text{if } u_k^- \leq u_{min} \text{ or } u_k^- \geq u_{max} \text{ then } (u_k - u_P) \cdot \frac{T_I}{k_P} \text{ else } I_k$ 
38:  end procedure

```

```

39:  procedure OBJECT.SAVE_STATE(date, filename) for OBJECT  $\in$  [header, follower]
40:      Save to CSV file  $I_k^+$ ,  $\chi_{\text{danger}}|_k$ 
41:  end procedure
42:  Select  $u_k = \text{if } \chi_{\text{danger}}|_k \text{ then } \text{follower}.u_k \text{ else } \text{header}.u_k$ 
43:  Compute  $u_{\text{actual}}|_k = u_0 + u_k$ 
44:  Convert  $u_{\text{actual}}|_k$  to on/off times for the PWM signal that controls the peristaltic pump
45:  Save results to CSV file. Plot and save to PNG files
46: end for

```

Algorithm 2 integrator.py

```

1: for  $k$  in range(0,  $T_c$ , end of experiment) do
2:   Initialize Logger
3:   procedure READ_ALL(file paths)
4:     Read all inputs: past  $u_{\text{actual}}$ ,  $\mathbf{d}_{\text{ref}}$  and  $\mathbf{x}$ 
5:     Read 'integrator.json': hyper-parameters for integration (start/end timestamps,  $\Delta_t$ )
6:     return  $u_{\text{actual}}$ ,  $\mathbf{d}_{\text{ref}}$ ,  $\mathbf{x}$ ,  $\theta_{\text{integrator}}$ 
7:   end procedure
8:   procedure PREPARE_MODEL_INPUTS( $u_{\text{actual}}$ ,  $\mathbf{d}_{\text{ref}}$ ,  $\mathbf{x}$ ,  $\theta_{\text{integrator}}$ )
9:     Filter on  $t \in [k, k-1]$  (from start to end timestamps) and do constant interpolation
10:    Use  $\theta_{\text{integrator}}$  to create  $\mathbf{T}$ 
11:    Evaluate quantities on  $\mathbf{T}$ 
12:    Extrapolate  $\mathbf{x}_0$  (at start timestamp)
13:    return  $\mathbf{T}$ ,  $\mathbf{x}_0$ ,  $\mathbf{d}_{\text{ref}}|_{\mathbf{T}}$ ,  $u|_{\mathbf{T}}$ 
14:  end procedure
15:  Save  $u|_{\mathbf{T}}$  and  $\mathbf{d}_{\text{ref}}|_{\mathbf{T}}$  to CombiTimeTable
16:  procedure MODELICA_INTEGRATOR.PY(model/outputs' names,  $\theta_{\text{integrator}}$ ,  $\mathbf{x}_0$ ,  $u_{\text{actual}}|_k$ )
17:    Create MOS file of the Modelica model called after input name: set simulation options,
     $\mathbf{x}_0$  and  $u_{\text{actual}}|_k$ .
18:    Call OpenModelica for integration via OMC compiler as a Python SUBPROCESS.
19:    Create and check LOG file.
20:    Extract outputs of interest from .mat file with the Python DYMAT library.
21:    return  $\mathbf{x}$ ,  $\tilde{\mathbf{y}}$ .
22:  end procedure
23:  Add Gaussian noise to  $\tilde{\mathbf{y}}$ .
24:  Save  $\tilde{\mathbf{y}}$  and  $\mathbf{x}|_k$  (to initialize real plant at  $k+1$ ).
25: end for

```

Algorithm 1 main.py (simplified)

- 1: Initialize Logger
 - 2: Round to HH:00:00 the current control run timestamp
 - 3: Read from CSV files *all* past *online* measurement data (alredy pre-processed with outlier detection and moving average for the total biogas flow rate). Cut between start/end timestamps.
 - 4: Compute methane flow rate and biogas compositon ratio from data. Extract the values corresponding to the current control run timestamp
 - 5: Read from TXT file the outputs' setpoints and the activation thresholds of the hysteresis logic
 - 6: Save measured and setpoint values at the current control run timestamp to CSV for checking and plotting purposes
 - 7: Compute the feedback errors
 - 8: Instantiate the PICONROLLER **class** of the SELECTORPI_CONTROLLER.PY file as *header* ($\rightarrow$ methane flow rate PI controller) and *follower* ($\rightarrow$ biogas ratio PI controller)
 - 9: Read from CSV file: (i) the values of the PI integrators' states and (ii) the boolean condition that indicates too high/unsafe values of the biogas ratio, at the previous control run timestamp
 - 10: Instantiate the HYSTERESISCOMPARATOR **class** of the SELECTORPI_CONTROLLER.PY file
 - 11: **if** Biogas ratio > 'safety' threshold **then**
 - 12: Set boolean condition to true i.e. activate *follower*
 - 13: **end if**
 - 14: **if** *follower* was activated now **then**
 - 15: Update the integral term of the *follower* to be consistent with the current control action
 - 16: **else if** *header* was activated now **then**
 - 17: Update the integral term of the *header* to be consistent with the current control action
 - 18: **end if**
 - 19: For both controllers compute the new control action's correction/deviation as a standard PI with saturation and anti-windup
 - 20: Save results to CSV files
 - 21: Select the the new control action's correction/deviation of the active controller
 - 22: Add the equilibrium/feedforward input to the correction/deviation computed by the controller to obtain the actual actuator command.
 - 23: Convert the value of the actual actuator command to *on/off* times for the PWM signal that controls the peristaltic pump
 - 24: Plot results and save to PNG files
-

Algorithm 2 integrator.py (simplified)

- 1: Initialize Logger.
 - 2: Read from CSV files *all* past control actions, flow rates of non-controlled feedstocks $\rightarrow$ feedstocks flow rates.
 - 3: Read from CSV files past states of the real plant.
 - 4: Read 'integrator.json': hyper-parameters for integration (start/end timestamps, step).
 - 5: Cut feedstocks flow rates between start/end timestamp, interpolate them and evaluate over the time grid defined by the integration hyper-parameters.
 - 6: Extrapolate the state at the start timestamp to initialize the integrator.
 - 7: Save feedstocks flow rates to **CombiTimeTable** to be read by OpenModelica.
 - 8: Call OpenModelica for integration (create MOS file) $\rightarrow$ LOG file. Set past control action and initial states $\rightarrow$ *online* measurable data and states.
 - 9: Add Gaussian noise to *online* measurable data.
 - 10: Save *online* measurable data and states at end timestamp (to initialize real plant at the next control run).
-

Actually, these functions are called at each control step k by the 'test_closedloop.ipynb' file, so that in practice the **for** loop is not present in the 'RPI_selectorPI_operative.py' and 'test_closedloop_integrator.py' files. Note: at initialization, $I_0 = 0$ and $\chi_{danger}|_0$ is false.